

ROS 2 Comprehensive Guide

Complete Documentation of Libraries, Classes, and
Implementation

Version 2.0 - 2024

AI Assistant

August 14, 2025

Contents

Preface	v
About ROS 2	vii
0.1 Key Features	vii
1 Core Architecture and Concepts	1
1.1 ROS 2 Architecture Overview	1
1.2 Core Concepts	1
1.2.1 Nodes	1
1.2.2 Topics	2
1.2.3 Services	2
1.2.4 Actions	2
2 Client Libraries	3
2.1 rclcpp (C++ Client Library)	3
2.1.1 Overview	3
2.1.2 Core Classes	3
2.1.3 Example: Simple Publisher	3
2.1.4 Example: Simple Subscriber	4
2.2 rclpy (Python Client Library)	5
2.2.1 Overview	5
2.2.2 Core Classes	5
2.2.3 Example: Simple Publisher	6
2.2.4 Example: Simple Subscriber	7
3 Messaging and Communication	9
3.1 Message Types	9
3.1.1 Standard Message Types	9
3.2 Services	9
3.2.1 Example: Service Server (C++)	9
3.2.2 Example: Service Client (C++)	10
4 Actions	13
4.1 Action Structure	13
4.2 Example: Fibonacci Action Server (Python)	13
5 Quality of Service (QoS)	15
5.1 QoS Policies	15
5.2 Example: Custom QoS (C++)	15

6	Parameters	17
6.1	Parameter Types	17
6.2	Example: Parameter Node (Python)	17
7	Launch Files	19
7.1	Python Launch Files	19
8	Package Structure and Build System	21
8.1	Package Structure	21
8.2	package.xml	21
9	Advanced Topics	23
9.1	Custom Message Types	23
9.1.1	Defining Custom Messages	23
9.1.2	Using Custom Messages (C++)	23
9.2	Node Composition	24
10	Tools and Utilities	25
10.1	Command-Line Tools	25
10.2	rqt Tools	25
11	Best Practices and Design Patterns	27
11.1	Node Design Principles	27
11.2	Performance Considerations	27
11.2.1	Message Size	27
11.2.2	Publishing Frequency	27
11.2.3	Memory Management	27
12	Debugging and Testing	29
12.1	Logging	29
12.2	Unit Testing	29
12.2.1	C++ Testing with gtest	29
12.2.2	Python Testing with pytest	30
13	Migration from ROS 1	31
13.1	Key Differences	31
13.2	Migration Checklist	31
14	Future Roadmap and Ecosystem	33
14.1	ROS 2 Roadmap	33
14.2	Ecosystem Packages	33
A	Message Reference	35
A.1	geometry_msgs	35
A.1.1	Twist Message	35
A.1.2	Pose Message	35
B	Service Reference	37
B.1	example_interfaces	37
B.1.1	AddTwoInts Service	37

C Build System Reference	39
C.1 CMakeLists.txt Template	39
C.2 setup.py Template	39
Glossary	41
Bibliography	43

List of Figures

1.1 ROS 2 Architecture Layers	1
1.2 Topic Communication Pattern	2
4.1 Action Communication Pattern	13

List of Tables

1.1 Node Properties	2
2.1 Core rclcpp Classes	3
2.2 Core rclpy Classes	6
3.1 Common ROS 2 Message Types	9
5.1 QoS Policies	15
10.1 Common ROS 2 Commands	25
13.1 ROS 1 vs ROS 2 Comparison	31
14.1 Recent ROS 2 Releases	33

Preface

This comprehensive guide provides detailed documentation of ROS 2 (Robot Operating System 2) libraries, classes, structures, usage patterns, syntax, and practical examples. ROS 2 represents a significant evolution from ROS 1, offering improved performance, real-time capabilities, security, and multi-platform support.

This document serves as both a reference manual and a practical guide for developers working with ROS 2, covering everything from basic concepts to advanced implementation patterns. Each chapter includes detailed examples, expected outputs, architectural diagrams, and best practices.

About ROS 2

ROS 2 is an open-source framework for developing robot applications. It provides a structured communications layer above the host operating systems of a heterogeneous compute cluster.

0.1 Key Features

- **Distributed Architecture:** Nodes can run on different machines
- **Real-time Support:** Built-in support for real-time systems
- **Security:** Native security features including authentication and encryption
- **Multi-platform:** Supports Linux, Windows, and macOS
- **Language Support:** C++, Python, and other language bindings
- **DDS Integration:** Uses Data Distribution Service (DDS) as middleware

Chapter 1

Core Architecture and Concepts

1.1 ROS 2 Architecture Overview

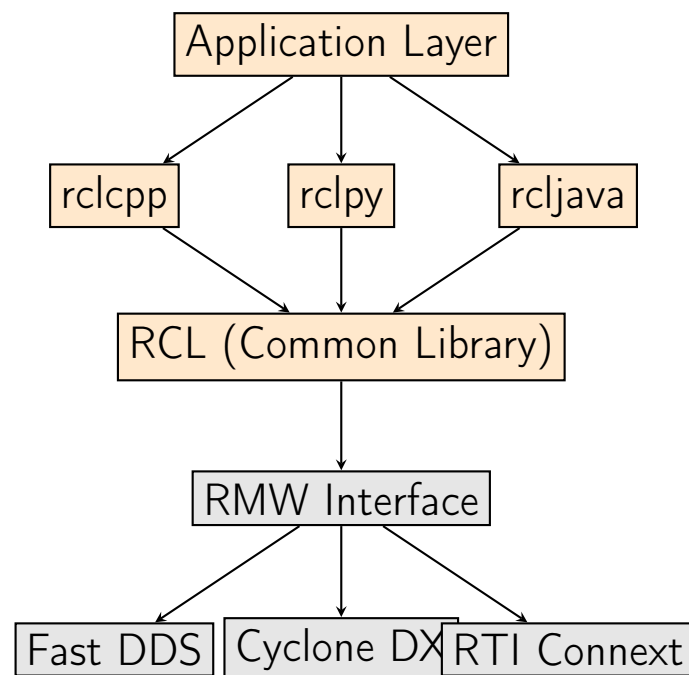


Figure 1.1: ROS 2 Architecture Layers

1.2 Core Concepts

1.2.1 Nodes

A **node** is a fundamental building block of ROS 2. It represents a single, modular purpose (e.g., controlling wheel motors, planning a path, etc.).

Property	Description
Name	Unique identifier within the ROS graph
Namespace	Hierarchical organization of nodes
Publishers	Send data on specific topics
Subscribers	Receive data from specific topics
Services	Provide request-response functionality
Actions	Handle long-running tasks with feedback
Parameters	Runtime configuration values

Table 1.1: Node Properties

1.2.2 Topics

Topics are named buses over which nodes exchange messages. They implement a publish-subscribe pattern.

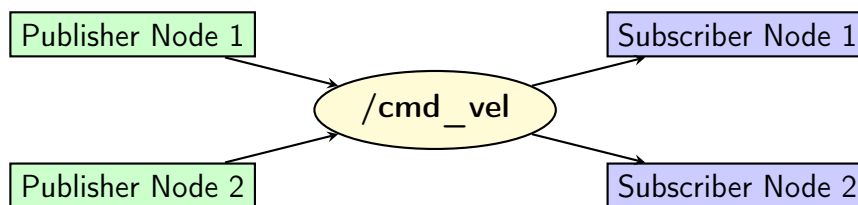


Figure 1.2: Topic Communication Pattern

1.2.3 Services

Services provide synchronous request-response communication between nodes.

1.2.4 Actions

Actions are used for long-running tasks that need feedback and can be preempted.

Chapter 2

Client Libraries

2.1 rclcpp (C++ Client Library)

2.1.1 Overview

rclcpp is the C++ client library for ROS 2, providing object-oriented APIs for creating nodes, publishers, subscribers, services, and actions.

2.1.2 Core Classes

Class	Purpose	Key Methods
rclcpp::Node	Base class for ROS 2 nodes	create_publisher(), create_subscription(), create_service(), create_client()
rclcpp::Publisher	Publishes messages to topics	publish(), get_subscription_count()
rclcpp::Subscriber	Receives messages from topics	Callback function, get_publisher_count()
rclcpp::Service	Provides service functionality	Callback function
rclcpp::Client	Calls services	async_send_request(), wait_for_service()
rclcpp::Timer	Periodic execution	cancel(), reset()
rclcpp::Executor	Manages node execution	spin(), spin_once(), add_node()

Table 2.1: Core rclcpp Classes

2.1.3 Example: Simple Publisher

```
1 #include <chrono>
2 #include <memory>
3 #include "rclcpp/rclcpp.hpp"
4 #include "std_msgs/msg/string.hpp"
5
6 using namespace std::chrono_literals;
```

```

7
8 class MinimalPublisher : public rclcpp::Node
9 {
10 public:
11     MinimalPublisher() : Node("minimal_publisher"), count_(0)
12     {
13         publisher_ = this->create_publisher<std_msgs::msg::String>(
14             "topic", 10);
15         timer_ = this->create_wall_timer(
16             500ms, std::bind(&MinimalPublisher::timer_callback, this));
17     }
18
19 private:
20     void timer_callback()
21     {
22         auto message = std_msgs::msg::String();
23         message.data = "Hello, world! " + std::to_string(count_++);
24         RCLCPP_INFO(this->get_logger(), "Publishing: '%s'",
25             message.data.c_str());
26         publisher_->publish(message);
27     }
28
29     rclcpp::TimerBase::SharedPtr timer_;
30     rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
31     size_t count_;
32 };
33
34 int main(int argc, char * argv[])
35 {
36     rclcpp::init(argc, argv);
37     rclcpp::spin(std::make_shared<MinimalPublisher>());
38     rclcpp::shutdown();
39     return 0;
40 }

```

Listing 2.1: Simple Publisher in C++

Expected Output

```

[INFO] [1234567890.123456789] [minimal_publisher]: Publishing: 'Hello, world! 0'
[INFO] [1234567890.623456789] [minimal_publisher]: Publishing: 'Hello, world! 1'
[INFO] [1234567891.123456789] [minimal_publisher]: Publishing: 'Hello, world! 2'
[INFO] [1234567891.623456789] [minimal_publisher]: Publishing: 'Hello, world! 3'
...

```

2.1.4 Example: Simple Subscriber

```

1 #include <memory>
2 #include "rclcpp/rclcpp.hpp"
3 #include "std_msgs/msg/string.hpp"
4
5 class MinimalSubscriber : public rclcpp::Node
6 {
7 public:
8     MinimalSubscriber() : Node("minimal_subscriber")
9     {

```

```

10     subscription_ = this->create_subscription<std_msgs::msg::String>(
11         "topic", 10,
12         std::bind(&MinimalSubscriber::topic_callback, this,
13                 std::placeholders::_1));
14     }
15
16 private:
17     void topic_callback(const std_msgs::msg::String::SharedPtr msg) const
18     {
19         RCLCPP_INFO(this->get_logger(), "I heard: '%s'", msg->data.c_str());
20     }
21
22     rclcpp::Subscription<std_msgs::msg::String>::SharedPtr subscription_;
23 };
24
25 int main(int argc, char * argv[])
26 {
27     rclcpp::init(argc, argv);
28     rclcpp::spin(std::make_shared<MinimalSubscriber>());
29     rclcpp::shutdown();
30     return 0;
31 }

```

Listing 2.2: Simple Subscriber in C++

Expected Output

```

[INFO] [1234567890.123456789] [minimal_subscriber]: I heard: 'Hello, world! 0'
[INFO] [1234567890.623456789] [minimal_subscriber]: I heard: 'Hello, world! 1'
[INFO] [1234567891.123456789] [minimal_subscriber]: I heard: 'Hello, world! 2'
[INFO] [1234567891.623456789] [minimal_subscriber]: I heard: 'Hello, world! 3'
...

```

2.2 rclpy (Python Client Library)

2.2.1 Overview

rclpy is the Python client library for ROS 2, providing Pythonic APIs for ROS 2 functionality.

2.2.2 Core Classes

Class	Purpose	Key Methods
rclpy.node.Node	Base class for ROS 2 nodes	create_publisher(), create_subscription(), create_service(), create_client()
rclpy.publisher.Publisher	Publishes messages to topics	publish(), get_subscription_count()
rclpy.subscription.Subscription	Receives subscription messages from topics	Callback function

Class	Purpose	Key Methods
<code>rclpy.service.Service</code>	Provides service functionality	Callback function
<code>rclpy.client.Client</code>	Calls services	<code>call_async()</code> , <code>wait_for_service()</code>
<code>rclpy.timer.Timer</code>	Periodic execution	<code>cancel()</code> , <code>reset()</code>

Table 2.2: Core rclpy Classes

2.2.3 Example: Simple Publisher

```

1 import rclpy
2 from rclpy.node import Node
3 from std_msgs.msg import String
4
5
6 class MinimalPublisher(Node):
7
8     def __init__(self):
9         super().__init__('minimal_publisher')
10        self.publisher_ = self.create_publisher(String, 'topic', 10)
11        timer_period = 0.5 # seconds
12        self.timer = self.create_timer(timer_period, self.timer_callback)
13
14        self.i = 0
15
16    def timer_callback(self):
17        msg = String()
18        msg.data = f'Hello World: {self.i}'
19        self.publisher_.publish(msg)
20        self.get_logger().info(f'Publishing: "{msg.data}"')
21        self.i += 1
22
23 def main(args=None):
24     rclpy.init(args=args)
25
26     minimal_publisher = MinimalPublisher()
27
28     rclpy.spin(minimal_publisher)
29
30     # Destroy the node explicitly
31     minimal_publisher.destroy_node()
32     rclpy.shutdown()
33
34
35 if __name__ == '__main__':
36     main()

```

Listing 2.3: Simple Publisher in Python

Expected Output

```

[INFO] [1234567890.123456789] [minimal_publisher]: Publishing: "Hello World: 0"
[INFO] [1234567890.623456789] [minimal_publisher]: Publishing: "Hello World: 1"
[INFO] [1234567891.123456789] [minimal_publisher]: Publishing: "Hello World: 2"

```

```
[INFO] [1234567891.623456789] [minimal_publisher]: Publishing: "Hello World: 3"
...
```

2.2.4 Example: Simple Subscriber

```
1 import rclpy
2 from rclpy.node import Node
3 from std_msgs.msg import String
4
5
6 class MinimalSubscriber(Node):
7
8     def __init__(self):
9         super().__init__('minimal_subscriber')
10        self.subscription = self.create_subscription(
11            String,
12            'topic',
13            self.listener_callback,
14            10)
15        self.subscription # prevent unused variable warning
16
17    def listener_callback(self, msg):
18        self.get_logger().info(f'I heard: "{msg.data}"')
19
20
21 def main(args=None):
22     rclpy.init(args=args)
23
24     minimal_subscriber = MinimalSubscriber()
25
26     rclpy.spin(minimal_subscriber)
27
28     # Destroy the node explicitly
29     minimal_subscriber.destroy_node()
30     rclpy.shutdown()
31
32
33 if __name__ == '__main__':
34     main()
```

Listing 2.4: Simple Subscriber in Python

Expected Output

```
[INFO] [1234567890.123456789] [minimal_subscriber]: I heard: "Hello World: 0"
[INFO] [1234567890.623456789] [minimal_subscriber]: I heard: "Hello World: 1"
[INFO] [1234567891.123456789] [minimal_subscriber]: I heard: "Hello World: 2"
[INFO] [1234567891.623456789] [minimal_subscriber]: I heard: "Hello World: 3"
...
```


Chapter 3

Messaging and Communication

3.1 Message Types

ROS 2 uses strongly-typed messages for communication. Standard message types are defined in the `std_msgs`, `geometry_msgs`, `sensor_msgs`, and other packages.

3.1.1 Standard Message Types

Package	Message Type	Description
<code>std_msgs</code>	<code>String</code>	Simple string message
<code>std_msgs</code>	<code>Int32</code>	32-bit integer
<code>std_msgs</code>	<code>Float64</code>	64-bit floating point
<code>geometry_msgs</code>	<code>Twist</code>	Linear and angular velocity
<code>geometry_msgs</code>	<code>Pose</code>	Position and orientation
<code>sensor_msgs</code>	<code>Image</code>	Camera image data
<code>sensor_msgs</code>	<code>LaserScan</code>	Laser range finder data

Table 3.1: Common ROS 2 Message Types

3.2 Services

Services provide synchronous request-response communication between nodes.

3.2.1 Example: Service Server (C++)

```
1 #include "rclcpp/rclcpp.hpp"
2 #include "example_interfaces/srv/add_two_ints.hpp"
3 #include <memory>
4
5 void add(const std::shared_ptr<example_interfaces::srv::AddTwoInts::
    Request> request,
6         std::shared_ptr<example_interfaces::srv::AddTwoInts::Response>
    response)
7 {
8     response->sum = request->a + request->b;
9     RCLCPP_INFO(rclcpp::get_logger("rclcpp"),
```

```

10         "Incoming request\na: %ld b: %ld", request->a, request->b)
11     ;
12     RCLCPP_INFO(rclcpp::get_logger("rclcpp"), "sending back response: [%ld
13         ]",
14         response->sum);
15 }
16
17 int main(int argc, char **argv)
18 {
19     rclcpp::init(argc, argv);
20
21     std::shared_ptr<rclcpp::Node> node =
22         rclcpp::Node::make_shared("add_two_ints_server");
23
24     rclcpp::Service<example_interfaces::srv::AddTwoInts>::SharedPtr
25         service =
26         node->create_service<example_interfaces::srv::AddTwoInts>(
27             "add_two_ints", &add);
28
29     RCLCPP_INFO(rclcpp::get_logger("rclcpp"), "Ready to add two ints.");
30
31     rclcpp::spin(node);
32     rclcpp::shutdown();
33 }

```

Listing 3.1: Service Server in C++

3.2.2 Example: Service Client (C++)

```

1 #include "rclcpp/rclcpp.hpp"
2 #include "example_interfaces/srv/add_two_ints.hpp"
3 #include <chrono>
4 #include <cstdlib>
5 #include <memory>
6
7 using namespace std::chrono_literals;
8
9 int main(int argc, char **argv)
10 {
11     rclcpp::init(argc, argv);
12
13     if (argc != 3) {
14         RCLCPP_INFO(rclcpp::get_logger("rclcpp"),
15             "usage: add_two_ints_client X Y");
16         return 1;
17     }
18
19     std::shared_ptr<rclcpp::Node> node =
20         rclcpp::Node::make_shared("add_two_ints_client");
21     rclcpp::Client<example_interfaces::srv::AddTwoInts>::SharedPtr client
22         =
23         node->create_client<example_interfaces::srv::AddTwoInts>("
24             add_two_ints");
25
26     auto request = std::make_shared<example_interfaces::srv::AddTwoInts::
27         Request>();
28     request->a = atoll(argv[1]);

```

```

26 request->b = atoll(argv[2]);
27
28 while (!client->wait_for_service(1s)) {
29     if (!rclcpp::ok()) {
30         RCLCPP_ERROR(rclcpp::get_logger("rclcpp"),
31                     "Interrupted while waiting for the service. Exiting."
32 );
33         return 0;
34     }
35     RCLCPP_INFO(rclcpp::get_logger("rclcpp"),
36                 "service not available, waiting again...");
37 }
38
39 auto result = client->async_send_request(request);
40 // Wait for the result.
41 if (rclcpp::spin_until_future_complete(node, result) ==
42     rclcpp::FutureReturnCode::SUCCESS)
43 {
44     RCLCPP_INFO(rclcpp::get_logger("rclcpp"), "Sum: %ld", result.get()->
45 sum);
46 } else {
47     RCLCPP_ERROR(rclcpp::get_logger("rclcpp"), "Failed to call service
48 add_two_ints");
49 }
50
51 rclcpp::shutdown();
52 return 0;
53 }

```

Listing 3.2: Service Client in C++

Expected Output

Server:

```

[INFO] [1234567890.123456789] [rclcpp]: Ready to add two ints.
[INFO] [1234567890.223456789] [rclcpp]: Incoming request
a: 2 b: 3
[INFO] [1234567890.223456789] [rclcpp]: sending back response: [5]

```

Client:

```

[INFO] [1234567890.223456789] [rclcpp]: Sum: 5

```


Chapter 4

Actions

Actions are used for long-running tasks that provide feedback and can be preempted.

4.1 Action Structure

Actions consist of three parts:

- **Goal**: What we want to achieve
- **Feedback**: Progress updates during execution
- **Result**: Final outcome of the action



Figure 4.1: Action Communication Pattern

4.2 Example: Fibonacci Action Server (Python)

```
1 import rclpy
2 from rclpy.action import ActionServer
3 from rclpy.node import Node
4 from action_tutorials_interfaces.action import Fibonacci
5
6
7 class FibonacciActionServer(Node):
8
9     def __init__(self):
10         super().__init__('fibonacci_action_server')
11         self._action_server = ActionServer(
12             self,
13             Fibonacci,
14             'fibonacci',
15             self.execute_callback)
16
17     def execute_callback(self, goal_handle):
18         self.get_logger().info('Executing goal...')
```

```
19
20     feedback_msg = Fibonacci.Feedback()
21     feedback_msg.partial_sequence = [0, 1]
22
23     for i in range(1, goal_handle.request.order):
24         feedback_msg.partial_sequence.append(
25             feedback_msg.partial_sequence[i] +
26             feedback_msg.partial_sequence[i-1])
27         self.get_logger().info(
28             f'Feedback: {feedback_msg.partial_sequence}')
29         goal_handle.publish_feedback(feedback_msg)
30
31         if goal_handle.is_cancel_requested:
32             goal_handle.canceled()
33             self.get_logger().info('Goal canceled')
34             return Fibonacci.Result()
35
36     goal_handle.succeed()
37
38     result = Fibonacci.Result()
39     result.sequence = feedback_msg.partial_sequence
40     return result
41
42
43 def main(args=None):
44     rclpy.init(args=args)
45
46     fibonacci_action_server = FibonacciActionServer()
47
48     rclpy.spin(fibonacci_action_server)
49
50
51 if __name__ == '__main__':
52     main()
```

Listing 4.1: Fibonacci Action Server in Python

Chapter 5

Quality of Service (QoS)

Quality of Service (QoS) policies allow you to configure the behavior of publishers and subscribers.

5.1 QoS Policies

Policy	Options	Description
Reliability	RELIABLE, BEST_EFFORT	Whether messages are guaranteed to arrive
Durability	VOLATILE, TRANSIENT_LOCAL	Whether messages are stored for late-joining subscribers
History	KEEP_LAST, KEEP_ALL	How many messages to keep in the queue
Depth	Integer	Number of messages to keep (when KEEP_LAST)
Deadline	Duration	Maximum time between messages
Lifespan	Duration	How long messages remain valid
Liveliness	AUTOMATIC, MANUAL_BY_TOPIC	How to determine if publishers are alive

Table 5.1: QoS Policies

5.2 Example: Custom QoS (C++)

```
1 #include "rclcpp/rclcpp.hpp"
2 #include "std_msgs/msg/string.hpp"
3
4 class CustomQoSSubscriber : public rclcpp::Node
5 {
6 public:
7     CustomQoSSubscriber() : Node("custom_qos_subscriber")
8     {
9         // Create custom QoS profile
```

```
10  auto qos = rclcpp::QoS(10) // Keep last 10 messages
11      .reliability(rclcpp::ReliabilityPolicy::Reliable)
12      .durability(rclcpp::DurabilityPolicy::TransientLocal)
13      .deadline(std::chrono::seconds(2))
14      .lifespan(std::chrono::seconds(5));
15
16  publisher_ = this->create_publisher<std_msgs::msg::String>(
17      "topic", qos);
18
19  timer_ = this->create_wall_timer(
20      std::chrono::milliseconds(1000),
21      std::bind(&CustomQoSSubscriber::timer_callback, this));
22  }
23
24 private:
25  void timer_callback()
26  {
27      auto message = std_msgs::msg::String();
28      message.data = "Hello with custom QoS!";
29      publisher_->publish(message);
30  }
31
32  rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
33  rclcpp::TimerBase::SharedPtr timer_;
34  };
```

Listing 5.1: Custom QoS Settings in C++

Chapter 6

Parameters

Parameters provide a way to configure nodes at runtime.

6.1 Parameter Types

ROS 2 supports the following parameter types:

- bool
- int64
- double
- string
- bytearray
- bool_array
- int64_array
- double_array
- string_array

6.2 Example: Parameter Node (Python)

```
1 import rclpy
2 from rclpy.node import Node
3 from rcl_interfaces.msg import ParameterDescriptor
4
5
6 class ParameterNode(Node):
7
8     def __init__(self):
9         super().__init__('parameter_node')
10
11         # Declare parameters with descriptions
12         my_parameter_descriptor = ParameterDescriptor(
13             description='This parameter controls the timer period.')
```

```
14         self.declare_parameter('my_parameter', 'world',
15         my_parameter_descriptor)
16         self.declare_parameter('timer_period', 1.0)
17
18         # Create timer that uses parameter
19         timer_period = self.get_parameter('timer_period').
get_parameter_value().double_value
20         self.timer = self.create_timer(timer_period, self.timer_callback
    )
21
22     def timer_callback(self):
23         my_param = self.get_parameter('my_parameter').
get_parameter_value().string_value
24         self.get_logger().info(f'Hello {my_param}!')
25
26         # You can also get all parameters
27         all_params = self.get_parameters(['my_parameter', 'timer_period'
    ])
28
29         for param in all_params:
30             self.get_logger().info(f'{param.name}: {param.value}')
31
32 def main(args=None):
33     rclpy.init(args=args)
34     node = ParameterNode()
35     rclpy.spin(node)
36     rclpy.shutdown()
37
38
39 if __name__ == '__main__':
40     main()
```

Listing 6.1: Parameter Usage in Python

Chapter 7

Launch Files

Launch files provide a way to start multiple nodes and configure the system.

7.1 Python Launch Files

```
1 from launch import LaunchDescription
2 from launch_ros.actions import Node
3 from launch.actions import DeclareLaunchArgument
4 from launch.substitutions import LaunchConfiguration
5
6
7 def generate_launch_description():
8     # Declare launch arguments
9     namespace_arg = DeclareLaunchArgument(
10         'namespace',
11         default_value='my_robot',
12         description='Namespace for the nodes'
13     )
14
15     use_sim_time_arg = DeclareLaunchArgument(
16         'use_sim_time',
17         default_value='false',
18         description='Use simulation time'
19     )
20
21     # Launch nodes
22     talker_node = Node(
23         package='demo_nodes_cpp',
24         executable='talker',
25         name='talker',
26         namespace=LaunchConfiguration('namespace'),
27         parameters=[{
28             'use_sim_time': LaunchConfiguration('use_sim_time')
29         }],
30         remappings=[
31             ('chatter', 'my_topic')
32         ]
33     )
34
35     listener_node = Node(
36         package='demo_nodes_py',
37         executable='listener',
```

```
38     name='listener',
39     namespace=LaunchConfiguration('namespace'),
40     parameters=[{
41         'use_sim_time': LaunchConfiguration('use_sim_time')
42     }],
43     remappings=[
44         ('chatter', 'my_topic')
45     ]
46 )
47
48 return LaunchDescription([
49     namespace_arg,
50     use_sim_time_arg,
51     talker_node,
52     listener_node
53 ])
```

Listing 7.1: Python Launch File Example

Chapter 8

Package Structure and Build System

8.1 Package Structure

A typical ROS 2 package has the following structure:

```
1 my_package/  
2 |-- CMakeLists.txt           (for C++ packages)  
3 |-- setup.py                (for Python packages)  
4 |-- setup.cfg                (for Python packages)  
5 |-- package.xml  
6 |-- src/  
7 |   +-- my_package/         (Python source code)  
8 |-- include/  
9 |   +-- my_package/         (C++ headers)  
10 |-- launch/  
11 |   +-- my_launch.py  
12 |-- config/  
13 |   +-- params.yaml  
14 |-- msg/  
15 |   +-- MyMessage.msg  
16 |-- srv/  
17 |   +-- MyService.srv  
18 |-- action/  
19 |   +-- MyAction.action  
20 +-- README.md
```

Listing 8.1: Package Structure

8.2 package.xml

```
1 <?xml version="1.0"?>  
2 <?xml-model href="http://download.ros.org/schema/package_format3.xsd"  
3   schematypens="http://www.w3.org/2001/XMLSchema"?>  
4 <package format="3">  
5   <name>my_package</name>  
6   <version>0.0.1</version>  
7   <description>Example ROS 2 package</description>  
8   <maintainer email="user@example.com">User Name</maintainer>  
9   <license>Apache-2.0</license>  
10  
11   <buildtool_depend>ament_cmake</buildtool_depend>
```

```
12
13 <depend>rclcpp</depend>
14 <depend>std_msgs</depend>
15 <depend>geometry_msgs</depend>
16
17 <test_depend>ament_lint_auto</test_depend>
18 <test_depend>ament_lint_common</test_depend>
19
20 <export>
21   <build_type>ament_cmake</build_type>
22 </export>
23 </package>
```

Listing 8.2: Example package.xml

Chapter 9

Advanced Topics

9.1 Custom Message Types

9.1.1 Defining Custom Messages

```
1 string first_name
2 string last_name
3 uint8 age
4 float64 height
5 bool is_student
```

Listing 9.1: Custom Message Definition (PersonMessage.msg)

9.1.2 Using Custom Messages (C++)

```
1 #include "rclcpp/rclcpp.hpp"
2 #include "my_package/msg/person_message.hpp"
3
4 class PersonPublisher : public rclcpp::Node
5 {
6 public:
7     PersonPublisher() : Node("person_publisher")
8     {
9         publisher_ = this->create_publisher<my_package::msg::PersonMessage>(
10             "person_info", 10);
11         timer_ = this->create_wall_timer(
12             std::chrono::seconds(1),
13             std::bind(&PersonPublisher::timer_callback, this));
14     }
15
16 private:
17     void timer_callback()
18     {
19         auto message = my_package::msg::PersonMessage();
20         message.first_name = "John";
21         message.last_name = "Doe";
22         message.age = 30;
23         message.height = 1.75;
24         message.is_student = false;
25
26         RCLCPP_INFO(this->get_logger(), "Publishing person info");
```

```

27     publisher_ ->publish(message);
28 }
29
30 rclcpp::Publisher<my_package::msg::PersonMessage>::SharedPtr
    publisher_;
31 rclcpp::TimerBase::SharedPtr timer_;
32 };

```

Listing 9.2: Using Custom Messages in C++

9.2 Node Composition

Node composition allows multiple nodes to run in the same process for better performance.

```

1 #include "rclcpp/rclcpp.hpp"
2 #include "rclcpp_components/register_node_macro.hpp"
3 #include "std_msgs/msg/string.hpp"
4
5 namespace composition
6 {
7
8 class Talker : public rclcpp::Node
9 {
10 public:
11     explicit Talker(const rclcpp::NodeOptions & options)
12         : Node("talker", options)
13     {
14         pub_ = create_publisher<std_msgs::msg::String>("chatter", 10);
15         timer_ = create_wall_timer(
16             std::chrono::milliseconds(1000),
17             std::bind(&Talker::on_timer, this));
18     }
19
20 private:
21     void on_timer()
22     {
23         auto msg = std::make_unique<std_msgs::msg::String>();
24         msg->data = "Hello World: " + std::to_string(count_++);
25         RCLCPP_INFO(this->get_logger(), "Publishing: '%s'", msg->data.c_str());
26         pub_ ->publish(std::move(msg));
27     }
28
29     rclcpp::Publisher<std_msgs::msg::String>::SharedPtr pub_;
30     rclcpp::TimerBase::SharedPtr timer_;
31     size_t count_ = 0;
32 };
33
34 } // namespace composition
35
36 RCLCPP_COMPONENTS_REGISTER_NODE(composition::Talker)

```

Listing 9.3: Composable Node Example

Chapter 10

Tools and Utilities

10.1 Command-Line Tools

ROS 2 provides many command-line tools for debugging and introspection:

Command	Description
<code>ros2 node list</code>	List running nodes
<code>ros2 topic list</code>	List active topics
<code>ros2 topic echo /topic_name</code>	Display messages on a topic
<code>ros2 topic pub /topic_name msg_type data</code>	Publish to a topic
<code>ros2 service list</code>	List available services
<code>ros2 service call /service_name srv_type request</code>	Call a service
<code>ros2 param list</code>	List parameters
<code>ros2 param get /node_name param_name</code>	Get parameter value
<code>ros2 param set /node_name param_name value</code>	Set parameter value
<code>ros2 launch package_name launch_file.py</code>	Run launch file
<code>ros2 bag record /topic_name</code>	Record topic data
<code>ros2 bag play bag_file</code>	Replay recorded data

Table 10.1: Common ROS 2 Commands

10.2 rqt Tools

rqt provides graphical tools for ROS 2:

- `rqt_graph`: Visualize the node graph
- `rqt_plot`: Plot numeric data from topics
- `rqt_console`: View log messages
- `rqt_param`: Edit parameters graphically
- `rqt_service_caller`: Call services through GUI
- `rqt_topic`: Monitor topics

Chapter 11

Best Practices and Design Patterns

11.1 Node Design Principles

1. **Single Responsibility**: Each node should have one clear purpose
2. **Loose Coupling**: Minimize dependencies between nodes
3. **Interface Segregation**: Use specific message types rather than generic ones
4. **Composition over Inheritance**: Prefer composition for complex behaviors
5. **Error Handling**: Always handle errors gracefully

11.2 Performance Considerations

11.2.1 Message Size

- Keep messages as small as possible
- Use appropriate data types (don't use `double` when `float` suffices)
- Consider compression for large data

11.2.2 Publishing Frequency

- Match publishing frequency to actual needs
- Use timers for periodic publishing
- Consider QoS settings for reliability vs. performance trade-offs

11.2.3 Memory Management

- Use smart pointers in C++
- Avoid memory leaks in long-running nodes
- Consider zero-copy transport for large messages

Chapter 12

Debugging and Testing

12.1 Logging

ROS 2 provides structured logging with different severity levels:

```
1 RCLCPP_DEBUG(this->get_logger(), "Debug message");
2 RCLCPP_INFO(this->get_logger(), "Info message");
3 RCLCPP_WARN(this->get_logger(), "Warning message");
4 RCLCPP_ERROR(this->get_logger(), "Error message");
5 RCLCPP_FATAL(this->get_logger(), "Fatal message");
```

Listing 12.1: Logging in C++

```
1 self.get_logger().debug('Debug message')
2 self.get_logger().info('Info message')
3 self.get_logger().warn('Warning message')
4 self.get_logger().error('Error message')
5 self.get_logger().fatal('Fatal message')
```

Listing 12.2: Logging in Python

12.2 Unit Testing

12.2.1 C++ Testing with gtest

```
1 #include <gtest/gtest.h>
2 #include "rclcpp/rclcpp.hpp"
3 #include "my_package/my_node.hpp"
4
5 class TestMyNode : public ::testing::Test
6 {
7 protected:
8     void SetUp() override
9     {
10         rclcpp::init(0, nullptr);
11         node_ = std::make_shared<MyNode>();
12     }
13
14     void TearDown() override
15     {
16         rclcpp::shutdown();
```

```

17     }
18
19     std::shared_ptr<MyNode> node_;
20 };
21
22 TEST_F(TestMyNode, TestCalculation)
23 {
24     int result = node_->calculate(2, 3);
25     EXPECT_EQ(result, 5);
26 }
27
28 int main(int argc, char **argv)
29 {
30     ::testing::InitGoogleTest(&argc, argv);
31     return RUN_ALL_TESTS();
32 }

```

Listing 12.3: Unit Test Example in C++

12.2.2 Python Testing with pytest

```

1 import pytest
2 import rclpy
3 from my_package.my_node import MyNode
4
5
6 class TestMyNode:
7
8     @classmethod
9     def setup_class(cls):
10         rclpy.init()
11
12     @classmethod
13     def teardown_class(cls):
14         rclpy.shutdown()
15
16     def setup_method(self):
17         self.node = MyNode()
18
19     def teardown_method(self):
20         self.node.destroy_node()
21
22     def test_calculation(self):
23         result = self.node.calculate(2, 3)
24         assert result == 5
25
26     def test_message_processing(self):
27         # Test message processing logic
28         processed = self.node.process_message("test")
29         assert processed == "processed: test"

```

Listing 12.4: Unit Test Example in Python

Chapter 13

Migration from ROS 1

13.1 Key Differences

Aspect	ROS 1	ROS 2
Master	Required	Not required
Language Support	C++, Python	C++, Python, others
Real-time	Limited	Native support
Security	Basic	Advanced
Platform Support	Linux	Linux, Windows, macOS
Communication	TCP/UDP	DDS
Build System	catkin	ament

Table 13.1: ROS 1 vs ROS 2 Comparison

13.2 Migration Checklist

1. Update package.xml to format 3
2. Replace CMakeLists.txt with ament_cmake
3. Update include statements and namespaces
4. Replace rospy/roscpp with rclpy/rclcpp
5. Update launch files to Python format
6. Test QoS settings for reliability
7. Update parameter handling
8. Verify timing behavior

Chapter 14

Future Roadmap and Ecosystem

14.1 ROS 2 Roadmap

The ROS 2 project continues to evolve with regular releases:

Release	Date	Key Features
Galactic	May 2021	ROS 2 maturation
Humble	May 2022	LTS Release
Iron	May 2023	Improved performance
Jazzy	May 2024	Enhanced security

Table 14.1: Recent ROS 2 Releases

14.2 Ecosystem Packages

Popular packages in the ROS 2 ecosystem:

- **Navigation2**: Mobile robot navigation
- **MoveIt2**: Motion planning framework
- **Gazebo**: 3D robot simulation
- **RViz2**: 3D visualization tool
- **tf2**: Coordinate frame transformations
- **rosbag2**: Data recording and playback
- **ros2_control**: Real-time control framework
- **micro-ROS**: ROS 2 for microcontrollers

Appendix A

Message Reference

A.1 geometry_msgs

A.1.1 Twist Message

```
1 # This expresses velocity in free space broken into its linear and
   angular parts.
2 Vector3 linear
3   float64 x
4   float64 y
5   float64 z
6 Vector3 angular
7   float64 x
8   float64 y
9   float64 z
```

Listing A.1: geometry_msgs/Twist.msg

A.1.2 Pose Message

```
1 # Position
2 Point position
3   float64 x
4   float64 y
5   float64 z
6
7 # Orientation
8 Quaternion orientation
9   float64 x
10  float64 y
11  float64 z
12  float64 w
```

Listing A.2: geometry_msgs/Pose.msg

Appendix B

Service Reference

B.1 example_interfaces

B.1.1 AddTwoInts Service

```
1 int64 a
2 int64 b
3 ---
4 int64 sum
```

Listing B.1: example_interfaces/srv/AddTwoInts.srv

Appendix C

Build System Reference

C.1 CMakeLists.txt Template

```
1 cmake_minimum_required(VERSION 3.8)
2 project(my_package)
3
4 if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
5     add_compile_options(-Wall -Wextra -Wpedantic)
6 endif()
7
8 # find dependencies
9 find_package(ament_cmake REQUIRED)
10 find_package(rclcpp REQUIRED)
11 find_package(std_msgs REQUIRED)
12
13 # Create executable
14 add_executable(my_node src/my_node.cpp)
15 ament_target_dependencies(my_node rclcpp std_msgs)
16
17 # Install
18 install(TARGETS
19     my_node
20     DESTINATION lib/${PROJECT_NAME})
21
22 if(BUILD_TESTING)
23     find_package(ament_lint_auto REQUIRED)
24     ament_lint_auto_find_test_dependencies()
25 endif()
26
27 ament_package()
```

Listing C.1: CMakeLists.txt for C++ package

C.2 setup.py Template

```
1 from setuptools import setup
2
3 package_name = 'my_package'
4
5 setup(
```

```
6     name=package_name ,
7     version='0.0.1',
8     packages=[package_name],
9     data_files=[
10         ('share/ament_index/resource_index/packages',
11          ['resource/' + package_name]),
12         ('share/' + package_name, ['package.xml']),
13         ('share/' + package_name + '/launch', ['launch/my_launch.py']),
14         ('share/' + package_name + '/config', ['config/params.yaml']),
15     ],
16     install_requires=['setuptools'],
17     zip_safe=True,
18     maintainer='Your Name',
19     maintainer_email='your.email@example.com',
20     description='Package description',
21     license='Apache License 2.0',
22     tests_require=['pytest'],
23     entry_points={
24         'console_scripts': [
25             'my_node = my_package.my_node:main',
26         ],
27     },
28 )
```

Listing C.2: setup.py for Python package

Glossary

Action Long-running tasks with feedback and preemption capabilities

Client Library Language-specific API for ROS 2 functionality

DDS Data Distribution Service - middleware standard used by ROS 2

Executor Component that manages node execution and callback handling

Launch File Configuration file to start multiple nodes and set parameters

Message Data structure for communication between nodes

Node Fundamental building block of ROS 2 applications

Parameter Runtime configuration value for nodes

Publisher Sends messages to a topic

QoS Quality of Service - policies that control communication behavior

Service Synchronous request-response communication pattern

Subscriber Receives messages from a topic

Topic Named channel for asynchronous message passing

Workspace Directory containing ROS 2 packages

Bibliography

Bibliography

- [1] Open Robotics. *ROS 2 Documentation*. <https://docs.ros.org/en/humble/>
- [2] Dirk Thomas, et al. *ROS 2 Design Documentation*. <https://design.ros2.org/>
- [3] Object Management Group. *Data Distribution Service (DDS) Specification*. <https://www.omg.org/spec/DDS/>
- [4] Open Robotics. *ROS 2 Tutorials*. <https://docs.ros.org/en/humble/Tutorials.html>
- [5] Open Robotics. *rclcpp Documentation*. <https://docs.ros2.org/humble/api/rclcpp/>
- [6] Open Robotics. *rclpy Documentation*. <https://docs.ros2.org/humble/api/rclpy/>